

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 12.5+7.5

Date : 15/01/2025
Matière : Programmation OO Java
Durée : 90 minutes
Documents : Non autorisés
Nombre des pages : 4

Examen

Exercice 1

Nous souhaitons créer un système en Java pour modéliser le transport de marchandises. Les marchandises sont représentées par des instances de la classe *Marchandise*, tandis que le transport se fait par l'intermédiaire de cargaisons qui peuvent être de différents types (fluvial, routier, aérien). Chaque type de cargaison a des règles spécifiques concernant le coût du transport et la capacité maximale.

La classe *Marchandise* doit encapsuler le poids de la marchandise. Elle est définie selon le code suivant :

```
class Marchandise {  
    private int poids;           // Poids en kilogrammes  
    public Marchandise(int poids){  
        this.poids=poids ;  
    }  
    public int getPoids(){  
        return poids ;         // Retourne le poids en kg  
    }  
}
```

La classe abstraite **Cargaison** représente une cargaison générique. Elle possède des attributs protégés pour la distance et la charge, ainsi qu'un constructeur qui initialise l'attribut distance avec la valeur fournie en paramètre et fixe l'attribut charge à zéro. Elle inclut également des méthodes publiques pour ajouter une marchandise et calculer le coût du transport :

- **distance** : un entier qui représente la distance en kilomètres.
- **charge** : un entier qui représente le poids total des marchandises en kilogrammes.
- Méthode abstraite : **void ajouter(int poids)** qui ajoute une marchandise à la cargaison si cela est possible.
- Méthode abstraite : **double calculerCout()** qui retourne le coût du transport de la cargaison.

Nous distinguons trois types de cargaisons, en fonction du moyen de transport utilisé, comme indiqué dans le tableau suivant :

Type de cargaison	Condition	Coût du transport d'une cargaison
Fluviale	$\text{charge} \leq 30000 \text{ kg}$	$\text{distance} * \text{charge} / 100$
Routière	$\text{charge} \leq 38000 \text{ kg}$	$2 * \text{distance} * \text{charge} / 100$
Aérienne	$\text{charge} \leq 80000 \text{ kg}$	$4 * \text{distance} * \text{charge} / 100$

Nous proposons également d'utiliser l'interface **Transportable** qui définit des méthodes essentielles que chaque type de cargaison doit implémenter :

- Méthode abstraite : **void afficherDetails()** pour afficher les détails spécifiques à chaque type de cargaison, notamment la distance et la charge.
- Méthode par défaut : **void afficherType()** qui affiche "Type de cargaison".
- Méthode statique : **void afficherInfoGenerale()** qui affiche "Types de cargaisons disponibles : Fluviale, Routière, Aérienne".

De plus, nous introduisons un record nommé **ResultatTransport**, conçu pour encapsuler les résultats liés au type de cargaison, représenté par une chaîne de caractères en minuscules, ainsi que le montant du coût du transport calculé.

Questions :

1. Définir la classe abstraite **Cargaison**, qui doit permettre l'héritage uniquement aux sous-classes **CargaisonFluviale**, **CargaisonRoutiere** et **CargaisonAerienne**.

```
sealed abstract class Cargaison permits CargaisonFluviale, CargaisonRoutiere,
CargaisonAerienne {
    protected int distance;
    protected int charge;

    public Cargaison(int distance) {
        this.distance = distance;
        this.charge = 0;
    }

    public abstract void ajouter(int poids);
    public abstract double calculerCout();
}
```

2. Définir l'interface **Transportable**

```
interface Transportable {
    void afficherDetails();

    default void afficherType() {
        System.out.println("Type de cargaison");
    }

    static void afficherInfoGenerale() {
        System.out.println("Types de cargaisons disponibles : Fluviale, Routière,
Aérienne.");
    }
}
```

3. Définir les classes concrètes **CargaisonFluviale**, **CargaisonRoutiere** et **CargaisonAerienne**, qui héritent de la classe **Cargaison**, implémentent l'interface **Transportable**, et peuvent être étendues sans restriction. Il convient de noter que la

capacité maximale de chaque type de cargaison (en termes de charge) est déclarée comme une constante dans la classe correspondante.

non-sealed class CargaisonFluviale extends Cargaison implements Transportable

```
{  
    private static final int CAPACITE_MAXIMALE = 30000;  
  
    public CargaisonFluviale(int distance) {  
        super(distance);  
    }  
  
    public void ajouter(int poids) {  
        if (charge + poids <= CAPACITE_MAXIMALE) {  
            charge += poids;  
        } else {  
            System.out.println("Capacité maximale atteinte !");  
        }  
    }  
  
    public double calculerCout() {  
        return (distance * charge) / 100.0; // Coût fluvial  
    }  
  
    public void afficherDetails() {  
        System.out.println("Cargaison Fluviale – Distance : " + distance + " km,  
Charge : " + charge + " kg");  
    }  
}
```

non-sealed class CargaisonRoutiere extends Cargaison implements Transportable {

```
    private static final int CAPACITE_MAXIMALE = 38000;  
  
    public CargaisonRoutiere(int distance) {  
        super(distance);  
    }  
  
    public void ajouter(int poids) {  
        if (charge + poids <= CAPACITE_MAXIMALE) {  
            charge += poids;  
        } else {  
            System.out.println("Capacité maximale atteinte !");  
        }  
    }  
  
    public double calculerCout() {  
        return (2 * distance * charge) / 100.0; // Coût routier  
    }  
  
    public void afficherDetails() {  
        System.out.println("Cargaison Routière – Distance : " + distance + " km,  
Charge : " + charge + " kg");  
    }  
}
```

```
}  
}
```

non-sealed class CargaisonAerienne extends Cargaison implements Transportable {

```
    private static final int CAPACITE_MAXIMALE = 80000;
```

```
    public CargaisonAerienne(int distance) {  
        super(distance);  
    }
```

```
    public void ajouter(int poids) {  
        if (charge + poids <= CAPACITE_MAXIMALE) {  
            charge += poids;  
        } else {  
            System.out.println("Capacité maximale atteinte !");  
        }  
    }  
}
```

```
    public double calculerCout() {  
        return (4 * distance * charge) / 100.0; // Coût aérien  
    }
```

```
    public void afficherDetails() {  
        System.out.println("Cargaison Aérienne – Distance : " + distance + " km,  
        Charge : " + charge + " kg");  
    }  
}
```

4. Définir le record **ResultatTransport**

```
record ResultatTransport(String typeCargaison, double coutTotal) {}
```

5. Définir la classe principale **Transportation**, qui doit permettre de :

- Afficher une information générale sur les types de cargaisons disponibles.
- Créer une instance de chaque type de cargaison (fluviale, routière, aérienne) en fonction de deux paramètres (type de cargaison et distance) récupérés à partir de deux entrées clavier pendant l'exécution.
- Ajoutez des marchandises à la cargaison en parcourant les poids suivants, qui sont fournis comme arguments lors de l'exécution du programme : {10000, 4000, 2000, 11500}.
- Calculer le coût du transport de cette cargaison.
- Afficher les détails de cette cargaison, notamment le type et le coût, en utilisant le record ResultatTransport.

Remarque : La méthode **toLowerCase()** de la classe String permet de convertir tous les caractères d'une chaîne en minuscules.

```
import java.util.Scanner;
```

```
public class Transportation {
    public static void main(String[] args) {

        Transportable.afficherInfoGenerale();

        Scanner sc = new Scanner (System.in);
        System.out.println("Saisir la distance parcourue :");
        int distance = sc.nextInt();
        System.out.println("Saisir le type de cargaison :");
        String type = sc.nextLine().toLowerCase();
        sc.close();

        Cargaison cargaison;

        switch (type) {
            case "fluviale":
                cargaison = new CargaisonFluviale(distance);
                break;
            case "routiere":
                cargaison = new CargaisonRoutiere(distance);
                break;
            case "aerienne":
                cargaison = new CargaisonAerienne(distance);
                break;
            default:
                System.out.println("Type de cargaison inconnu");
                return;
        }
        // int[] t = {10000, 4000, 2000, 11500};
        // args= new String[t.length];
        // for (int i=0; i< t.length;i++) {
        //     args[i]=Integer.toString(t[i]);
        // }
        for (int i=0; i< args.length;i++) {
            cargaison.ajouter(Integer.parseInt(args[i]));
        }

        double coutTotal = cargaison.calculerCout();

        ResultatTransport details = new ResultatTransport(type, coutTotal);

        System.out.println("Type de cargaison : " + details.typeCargaison());
        System.out.println("Coût total du transport : " + details.coutTotal());

        // System.out.println("Charge totale :"+cargaison.charge+ " Distance : "
        // +cargaison.distance);

    }
}
```

Exercice 2

Supposons que le JDK 23 soit déjà installé et que les previews de cette version soient activées. Pour chaque exemple suivant, indiquez s'il est correct. Si c'est le cas, fournissez le résultat de l'exécution de la classe principale. Sinon, précisez s'il s'agit d'une erreur de compilation ou d'une erreur d'exécution, identifiez-la et proposez une correction.

1.

```
class A {
    public A() {
        System.out.println("Je suis dans A");
    }
}
class B extends A {

    public B() {
        System.out.println("B prologue");
        super();
        System.out.println("B epilogue");
    }
}
public class Test1 {
    public static void main(String[] args) {
        A obj =new B();
    }
}
```

Cet exemple est correct. Il affiche :
B prologue
Je suis dans A
B epilogue

2.

```
public class Test2 {
    public static int f(Integer... x) { return 1;}
    public static int f(Long x) {return 2;}
    public static int f(short x) {return 3;}
    public static void main(String[] args) {
        var val = 10;
        System.out.println(f(val));
    }
}
```

Cet exemple est correct. Il affiche :
1

3.

```
interface Operation {
    void affiche();
}
public class Test3 {
    public void printMessage() {
        System.out.println("Bienvenue");
    }
}
```

```

    }
    public static void main(String[] args) {
        Operation obj = Test3::printMessage;
        obj.affiche();
    }
}

```

Cet exemple est incorrect. Il y a une erreur de compilation au niveau de l'instruction `Operation obj = Test3::printMessage;` Il est impossible de proposer une référence à une méthode statique pour appeler la méthode d'instance `printMessage()`. Il faut mettre alors :

`Operation obj = new Test3().printMessage;`

4.

```

class X{
    public int f(int a) { return a++;}
}
class Y extends X{
    protected int f(int a , int b) { return a+b;}
}
public class Test4 {
    public static void main(String[] args) {

        X obj = new Y();
        System.out.println(obj.f(5));
    }
}

```

Cet exemple est correct. Il affiche :
5

5.

```

public class Test5 {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("EN");
        sb.append("CPI2").insert(1, "XAME");
        System.out.println(sb.capacity());
        System.out.println(sb.length());
        System.out.println(sb.toString());
    }
}

```

Cet exemple est correct. Il affiche :
18
10
EXAMENCPI2

6.

```

class A1 {
    public String f(B1 obj) { return "A1 et B1"; }
}
class B1 extends A1 {
    public String f(A1 obj) { return "B1 et A1"; }
}
interface X1{

```

```
    default String f(A1 obj) { return "X1 et A1"; }
}
interface Y1 extends X1{
    default String f(A1 obj) { return "Y1 et A1"; }
}
class C1 extends A1 implements Y1{
    public String f(A1 obj) { return Y1.super.f(obj); }
}
public class Test6 {
    public static void main(String[] args) {
        Y1 obj1 = new C1();
        System.out.println(obj1 instanceof X1);
        B1 obj2=(B1) obj1;
        System.out.println(obj2.f(new A1()));
    }
}
```

**Cet exemple est incorrect. Il y a une erreur d'exécution au niveau de l'instruction
B1 obj2=(B1) obj1; Il est impossible de caster l'objet obj1 avec la classe B1. Il
faut mettre alors :**

```
    A1 obj2=(A1) obj1;  
    System.out.println(obj2.f(new B1()));
```

Ou bien

```
    if(obj1 instanceof B1) {  
        B1 obj2=(B1) obj1;  
        System.out.println(obj2.f(new A1()));  
    }
```

Ou bien

```
class C1 extends B1 implements Y1{ ...}
```

BON TRAVAIL